

DOCUMENT 2

QUALIFYAI — FULL TECHNICAL IMPLEMENTATION PLAN

Project: QualifyAI

Document: Document 2 — Full Technical Implementation Plan

Document Type: Technical Implementation Specification

Status: Foundation Technical Blueprint

Version: 1.0

1. DOCUMENT OVERVIEW

This document defines the technical implementation plan for **QualifyAI**.

Document 1 established what QualifyAI does from a product and functional perspective. This document translates those requirements into an implementation-oriented technical structure.

The objective is to establish:

- application architecture,
- frontend implementation boundaries,
- backend implementation boundaries,
- API responsibilities,
- AI processing pipeline,
- real-time communication,
- database interaction,
- storage interaction,
- authentication,
- authorization,
- asynchronous processing,
- evaluation pipeline,
- proctoring architecture,
- reporting,
- security,

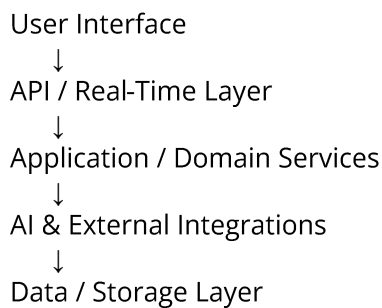
- performance,
- deployment,
- and development sequencing.

The implementation must remain consistent with the QualifyAI product definition.

2. IMPLEMENTATION PHILOSOPHY

QualifyAI should be implemented as a modular full-stack SaaS platform.

The core principle is:



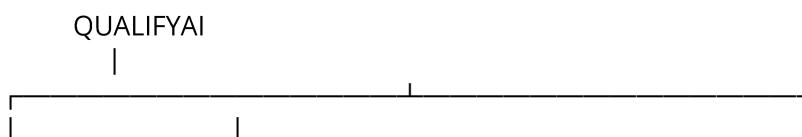
Each layer must have a clear responsibility.

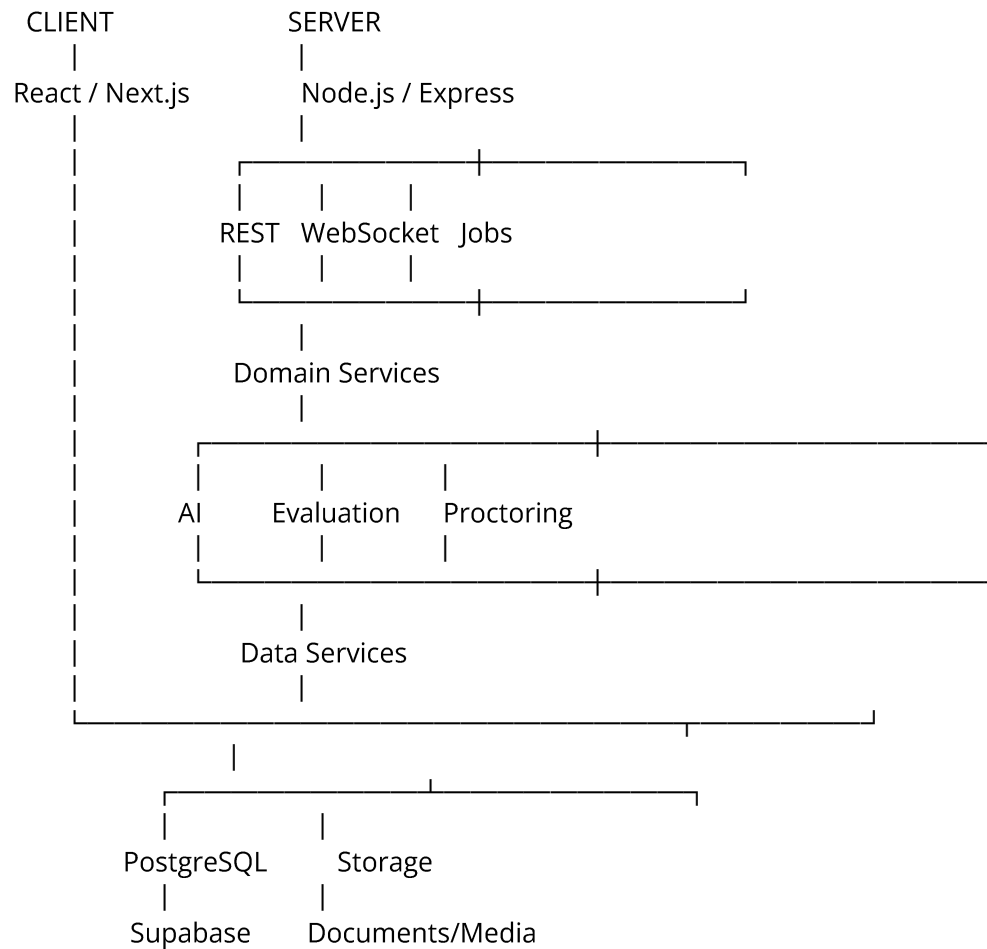
The implementation should avoid tightly coupling:

- UI to business logic,
- routes to database queries,
- AI providers to core business logic,
- storage implementation to frontend code,
- and evaluation logic to a single AI provider.

3. HIGH-LEVEL TECHNICAL ARCHITECTURE

The implementation can be represented as:





4. TECHNOLOGY STACK

The documented QualifyAI technology direction is:

4.1 Frontend

- Next.js 14
- React 18
- JavaScript
- Tailwind CSS
- Shadcn UI

4.2 Backend

- Node.js 20
- Express 4

4.3 Real-Time Communication

- WebSockets
- Socket.io

4.4 Database

- PostgreSQL
- Prisma 5

4.5 Authentication

- Supabase Auth
- JWT

4.6 Asynchronous Processing

- Redis
- BullMQ

4.7 Speech-to-Text

- Deepgram Nova-2

4.8 Large Language Model

The documented architecture supports:

- OpenAI GPT-4o
- Gemini-based LLM capabilities

4.9 Text-to-Speech

- ElevenLabs

4.10 Deployment Direction

- Docker
- AWS EC2

- Vercel

These technologies represent the documented implementation direction and may be refined during development.

5. REPOSITORY ARCHITECTURE

The project should maintain the following high-level structure:

```
QUALIFYAI/  
|  
├── README.md  
├── prd.md  
├── architecture.md  
├── design.md  
├── memory.md  
├── phases.md  
├── rules.md  
|  
├── client/  
├── server/  
├── storage/  
├── document/  
└── docs/
```

The root Markdown files are the permanent project source of truth.

The implementation is divided into:

client/
→ frontend application

server/
→ backend/API application

storage/
→ application file/media storage organization

document/
→ formal project artifacts and references

docs/
→ supporting technical engineering documentation

6. FRONTEND IMPLEMENTATION

The client application provides the user-facing interface for both major platform experiences.

```
client/
|
├── public/
|
└── src/
    ├── assets/
    ├── components/
    ├── layouts/
    ├── pages/
    ├── routes/
    ├── services/
    ├── hooks/
    ├── context/
    ├── lib/
    ├── constants/
    ├── utils/
    └── styles/
```

7. FRONTEND DOMAIN ORGANIZATION

The frontend should be organized around product domains.

Major areas include:

- Authentication
- Recruiter
- Candidate
- Jobs
- Candidates
- Interviews
- AI Interview
- Evaluation
- Reports
- Analytics
- Proctoring
- Shared UI

The interface should not be organized as one large collection of unrelated components.

Each major product domain should have a clear boundary.

8. FRONTEND RESPONSIBILITIES

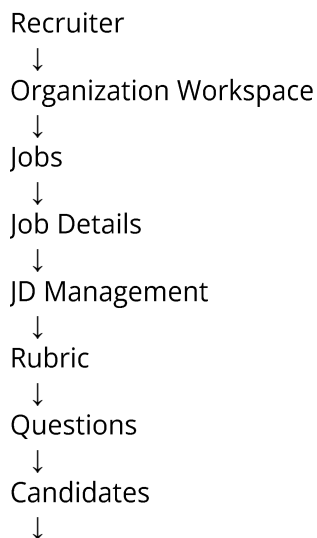
The frontend is responsible for:

- rendering UI,
- handling user interaction,
- managing client-side state,
- form handling,
- navigation,
- recruiter dashboards,
- candidate interview interface,
- voice interaction controls,
- report visualization,
- analytics visualization,
- responsive behavior,
- accessibility.

The frontend must not contain authoritative business rules that should be enforced by the backend.

9. RECRUITER FRONTEND

The recruiter application should provide interfaces for:



Interviews
↓
Evaluations
↓
Reports
↓
Analytics

Important recruiter views include:

- recruiter dashboard,
- job management,
- JD upload,
- requirement review,
- rubric review,
- question review,
- candidate list,
- interview status,
- candidate evaluation,
- transcript review,
- candidate comparison,
- ranking,
- reports.

10. CANDIDATE FRONTEND

The candidate application should focus on a low-friction interview experience.

Major areas include:

Candidate
↓
Invitation
↓
Interview Preparation
↓
System Check
↓
Interview Room
↓
AI Conversation
↓
Interview Completion
↓

Feedback

The interview interface should prioritize:

- clear instructions,
 - microphone status,
 - AI speaking status,
 - candidate speaking status,
 - question context,
 - interview progress,
 - connection status,
 - error recovery,
 - completion state.
-

11. AI INTERVIEW UI

The AI interview interface is a specialized real-time experience.

It must communicate the current system state.

Example states:



The interface should clearly communicate transitions without overwhelming the candidate.

12. BACKEND ARCHITECTURE

The backend is the primary application and orchestration layer.

```
server/  
├── src/  
│   ├── config/  
│   ├── constants/  
│   ├── middleware/  
│   ├── routes/  
│   ├── controllers/  
│   ├── services/  
│   ├── validators/  
│   ├── integrations/  
│   ├── modules/  
│   └── utils/
```

13. BACKEND RESPONSIBILITIES

The backend is responsible for:

- authentication enforcement,
 - authorization,
 - tenant isolation,
 - job management,
 - JD processing,
 - rubric generation,
 - question generation,
 - candidate management,
 - invitations,
 - interview orchestration,
 - real-time interview communication,
 - AI provider communication,
 - transcript processing,
 - evaluation,
 - scoring,
 - proctoring,
 - report generation,
 - database access,
 - storage authorization,
 - asynchronous processing.
-

14. API ARCHITECTURE

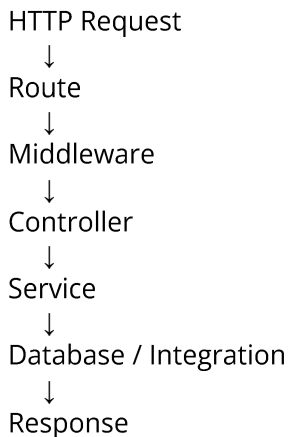
The API should use a structured REST architecture.

Conceptually:



Routes should remain thin.

The preferred request flow is:



15. CONTROLLER LAYER

Controllers should handle the HTTP boundary.

Their responsibilities include:

- receiving validated requests,

- invoking appropriate services,
- handling response formatting,
- returning appropriate HTTP responses.

Controllers should not contain large amounts of business logic.

16. SERVICE LAYER

The service layer contains business logic.

Examples include:

JD Service
Rubric Service
Question Service
Candidate Service
Interview Service
Evaluation Service
Scoring Service
Report Service
Proctoring Service
Analytics Service

Services should coordinate business operations without exposing provider-specific implementation details.

17. INTEGRATION LAYER

External systems should be isolated inside an integration boundary.

Conceptually:

integrations/
|
├── authentication/
├── ai/
├── speech/
├── storage/
├── notifications/
└── infrastructure/

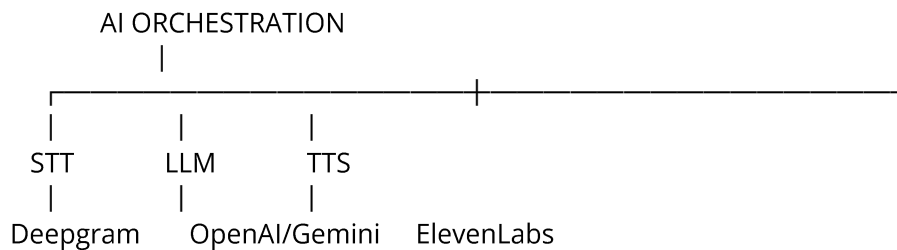
The exact implementation structure can evolve according to actual requirements.

The important rule is:

External provider logic must not be scattered throughout the application.

18. AI INTEGRATION ARCHITECTURE

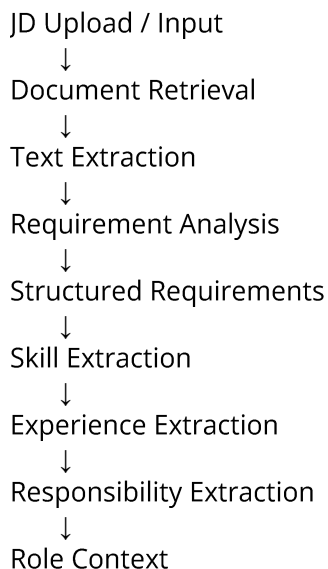
The AI architecture should use provider abstraction.



This allows the platform to replace providers without rewriting the interview engine.

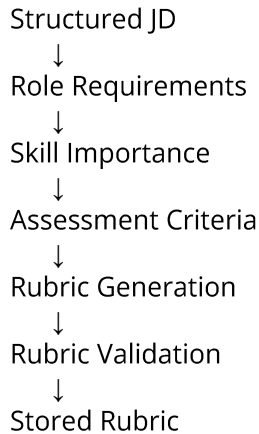
19. JOB DESCRIPTION PROCESSING PIPELINE

The JD pipeline should operate as:



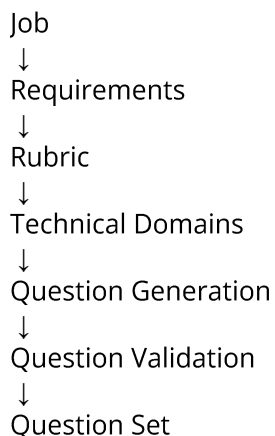
The resulting structured information becomes the foundation for assessment generation.

20. RUBRIC GENERATION PIPELINE



The rubric must remain associated with the specific job.

21. QUESTION GENERATION PIPELINE



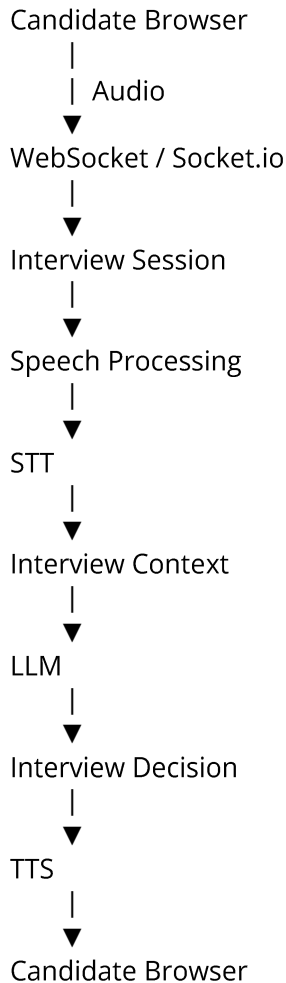
Questions should remain associated with:

- organization,
- job,
- rubric,
- interview configuration where applicable.

22. REAL-TIME INTERVIEW ARCHITECTURE

The real-time interview is one of the most technically sensitive components.

The communication path is:



The interview engine must maintain session state throughout the conversation.

23. INTERVIEW SESSION MANAGEMENT

Each interview should have a server-side session context.

The context should track:

- interview ID,
- candidate,
- job,
- rubric,
- question plan,
- current question,
- previous responses,
- transcript,
- evaluation state,
- interview status,
- integrity events.

The session must be securely associated with the authorized candidate.

24. SPEECH-TO-TEXT IMPLEMENTATION

The STT layer should:

1. Receive candidate audio.
2. Stream/process audio.
3. Produce transcription.
4. associate transcription with the current interview turn.
5. persist relevant transcript data.
6. pass the result to the interview engine.

The implementation should support low-latency processing.

The documented target is approximately sub-300ms STT processing within the overall latency budget.

25. LLM PROCESSING

The LLM layer should receive structured interview context rather than only the latest candidate sentence.

Context can include:

Job Description
+
Rubric
+
Current Question
+
Previous Questions
+
Previous Responses
+
Interview State

The LLM can then determine:

- whether the response is relevant,
- whether deeper probing is appropriate,
- whether clarification is required,
- what question should come next,
- and what evaluation signals may be relevant.

26. TEXT-TO-SPEECH IMPLEMENTATION

The TTS layer receives the AI-generated response.

LLM Output
↓
Response Validation
↓
TTS
↓
Audio Stream
↓
Candidate

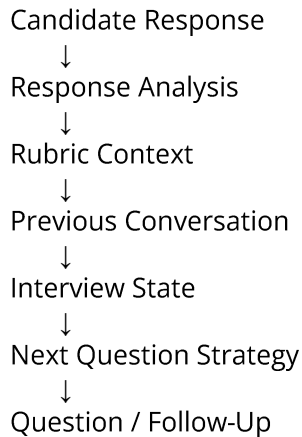
The TTS service should remain independent from the core interview engine.

The documented target is approximately 350ms within the overall voice-latency budget.

27. ADAPTIVE QUESTIONING ENGINE

The adaptive interview engine should evaluate the conversation state before selecting the next question.

Conceptually:



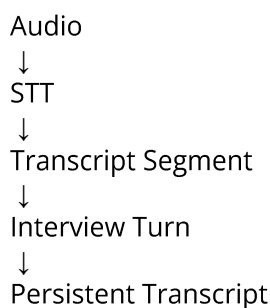
Possible strategies include:

- continue planned question,
- ask clarification,
- probe deeper,
- test practical understanding,
- move to another competency.

The actual strategy must remain controlled by the interview configuration and rubric.

28. TRANSCRIPT PIPELINE

The transcript pipeline should maintain the interview conversation.



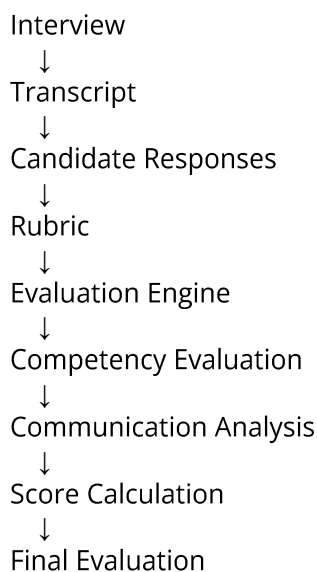
The transcript should distinguish between:

- AI speech,
- candidate speech.

It should also preserve interview ordering.

29. EVALUATION PIPELINE

Evaluation should occur through a structured pipeline.



The evaluation engine should remain separate from the conversational interview engine.

30. SCORING ENGINE

The scoring engine converts evaluation information into structured scores.

The documented system uses a 0–100 scoring direction.

The implementation should support:

- competency scores,
- overall score,
- communication metrics,

- rubric-level results,
- supporting evaluation information.

Scores should remain traceable to their underlying evaluation context.

31. COMMUNICATION METRICS

The system can calculate communication metrics including:

- words per minute,
- filler-word density,
- communication clarity.

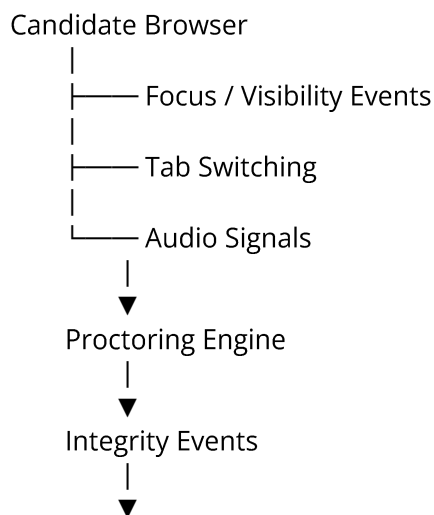
These metrics should be derived from interview transcript/audio data where technically supported.

They should be presented as assessment signals rather than treated as the sole measure of candidate quality.

32. PROCTORING ENGINE

The proctoring system operates independently from technical evaluation.

Conceptually:



The system should record relevant events with timestamps and interview association.

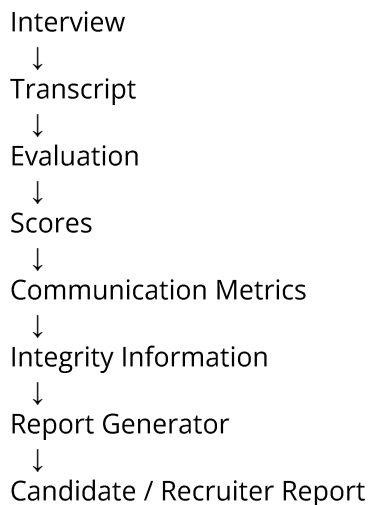
33. PROCTORING BOUNDARIES

The implementation must not introduce facial-expression or micro-expression scoring as a core evaluation mechanism.

The integrity system should focus on observable assessment-integrity signals rather than attempting to infer candidate personality or intent from facial expressions.

34. REPORT GENERATION

The report pipeline should be:



The platform should support a diagnostic PDF report according to the documented product direction.

35. ASYNCHRONOUS PROCESSING

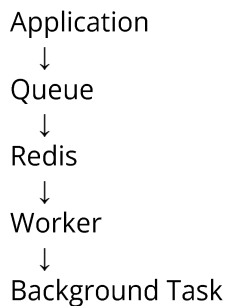
Some operations should not block the primary user request.

Potential asynchronous tasks include:

- report generation,
- heavy document processing,
- transcript post-processing,
- analytics aggregation,
- notification processing,
- background evaluation tasks.

BullMQ and Redis are part of the documented technology direction for such workloads.

Conceptually:

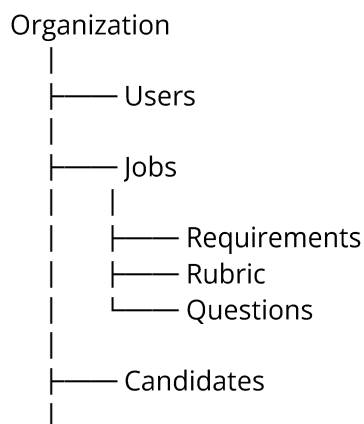


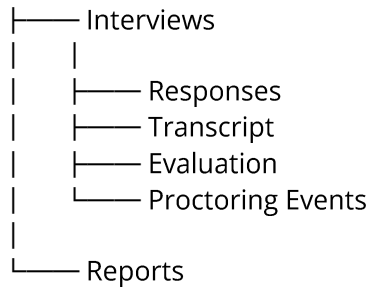
36. DATABASE IMPLEMENTATION

The database uses PostgreSQL with Prisma.

The database layer should represent core QualifyAI relationships.

Conceptually:



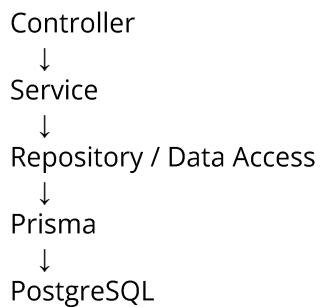


The exact schema should be finalized during the database architecture phase.

37. PRISMA DATA ACCESS

Application services should access PostgreSQL through Prisma rather than embedding raw database logic throughout the codebase.

Preferred flow:

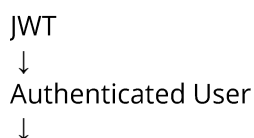


The exact repository abstraction should be introduced only where it provides meaningful separation.

38. MULTI-TENANT DATA FLOW

Every organization-owned request should establish tenant context.

Conceptually:



Organization Context
↓
Authorization
↓
Service
↓
Tenant-Scoped Query
↓
Database

A user must never be able to retrieve another organization's jobs, candidates, interviews, or reports by manipulating request parameters.

39. AUTHENTICATION FLOW

User
↓
Supabase Auth
↓
JWT
↓
Client
↓
API Request
↓
Authentication Middleware
↓
JWT Validation
↓
User / Organization Context
↓
Authorization
↓
Controller

The backend must independently validate authorization.

40. STORAGE IMPLEMENTATION

QualifyAI storage should be logically organized around application artifacts.

storage/
├── resumes/

- |— job-descriptions/
- |— interview-recordings/
- |— transcripts/
- |— reports/
- |— candidate-documents/
- |— organization-assets/
- |— temp/

Storage access should never be assumed to be public.

Access must be authorized based on:

- user,
- role,
- organization,
- resource ownership.

41. API SECURITY

API endpoints must implement:

- authentication,
- authorization,
- request validation,
- input sanitization,
- rate limiting,
- safe error handling.

Sensitive implementation details must not be returned to clients.

42. WEBSOCKET SECURITY

Real-time interview communication must also be authenticated.

The system should validate:

- candidate identity,
- interview identity,

- authorization,
- session state.

A candidate must not be able to connect to another candidate's interview by modifying a client-side identifier.

43. ERROR HANDLING

The backend should use consistent error handling.

Conceptually:

Request
↓
Validation Error
OR
Authentication Error
OR
Authorization Error
OR
Business Error
OR
Integration Error
OR
Unexpected Error

Errors should be:

- predictable,
- structured,
- safe,
- logged appropriately.

Internal stack traces and secrets must not be exposed to clients.

44. AI FAILURE HANDLING

External AI services can fail or become unavailable.

The interview engine must therefore account for:

- STT failure,
- LLM timeout,
- TTS failure,
- network interruption,
- malformed provider response,
- rate limits.

The system should provide controlled recovery mechanisms instead of leaving the interview in an undefined state.

45. INTERVIEW STATE MACHINE

The interview engine should maintain explicit interview states.

A conceptual state flow is:

```
graph TD
    CREATED --> INVITED
    INVITED --> READY
    READY --> IN_PROGRESS[IN_PROGRESS]
    IN_PROGRESS --> PROCESSING
    PROCESSING --> COMPLETED
    COMPLETED --> EVALUATING
    EVALUATING --> EVALUATED
    EVALUATED --> REPORT_READY[REPORT_READY]
```

Failure states should be represented explicitly rather than silently ignored.

46. REPORT ACCESS

Reports must be accessible only to authorized users.

Recruiter report access should be limited to the organization and job/candidate context to which the recruiter has access.

Candidate feedback should expose only the information intended for the candidate.

47. ANALYTICS IMPLEMENTATION

Analytics should be derived from authorized assessment data.

Potential aggregation layers include:

Interview Data
↓
Evaluation Data
↓
Score Data
↓
Aggregations
↓
Recruiter Dashboard

Analytics processing should not unnecessarily block interview completion.

48. PERFORMANCE ARCHITECTURE

Performance-sensitive areas include:

Real-Time Voice

Requires low-latency communication.

Interview Processing

Must avoid unnecessary synchronous operations.

Database

Requires appropriate indexes and efficient tenant-scoped queries.

Storage

Large media artifacts should not pass through the application server unnecessarily when direct authorized storage mechanisms are available.

Analytics

Heavy aggregation should use asynchronous processing where appropriate.

49. TARGET VOICE LATENCY

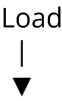
The documented architecture provides an approximate latency target:

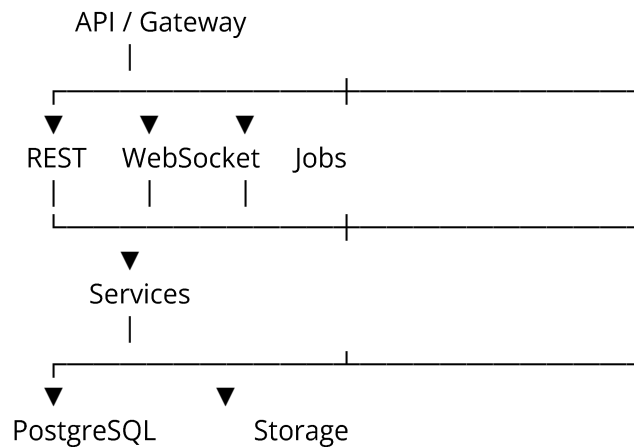


These are engineering targets and must be measured using real implementation conditions.

50. SCALABILITY ARCHITECTURE

The system should be designed so that major workloads can scale independently.





The project target includes support for 1,000+ simultaneous candidate assessment sessions.

This must be validated through load and stress testing.

51. OBSERVABILITY

The system should provide sufficient observability for:

- API failures,
- interview failures,
- AI provider failures,
- WebSocket issues,
- background job failures,
- database errors,
- storage failures.

Important interview lifecycle events should be traceable through a unique interview/session context.

52. LOGGING

Logs should be structured and useful for debugging.

Avoid logging:

- passwords,
- authentication secrets,
- private tokens,
- unnecessary candidate-sensitive information.

Logs should support tracing important operations without compromising privacy.

53. DEPLOYMENT ARCHITECTURE

The documented deployment direction includes:

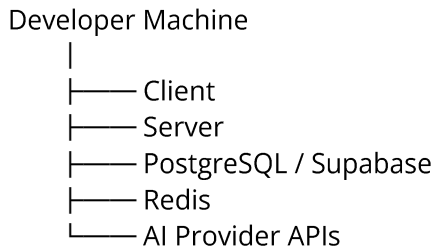


The exact production deployment configuration may evolve during implementation.

54. DEVELOPMENT ENVIRONMENT

Development should allow the major application layers to run independently.

Conceptually:



Environment variables must be separated appropriately between client and server.

Private credentials must remain server-side.

55. ENVIRONMENT CONFIGURATION

The project should support distinct environments such as:

Development
Testing
Production

Environment-specific configuration must not be hard-coded into application logic.

Sensitive values should be supplied through environment configuration/secrets.

56. TESTING STRATEGY

Testing should eventually cover:

Unit Testing

- services,
- utility functions,
- evaluation logic,
- scoring logic.

Integration Testing

- API,

- database,
- authentication,
- storage,
- AI integrations.

Real-Time Testing

- WebSocket connections,
- audio pipeline,
- interview state.

End-to-End Testing

- recruiter workflow,
- candidate workflow,
- interview completion,
- report generation.

Performance Testing

- voice latency,
- concurrent interviews,
- API throughput,
- database performance.

57. AI TESTING

AI components should not be tested only through subjective manual inspection.

Testing should include:

- known input/output cases,
- structured output validation,
- rubric consistency,
- question relevance,
- malformed responses,
- provider failures,
- latency,
- prompt regression.

AI behavior should remain constrained by explicit application rules.

58. SECURITY TESTING

Security testing should cover:

- authentication bypass,
 - authorization bypass,
 - tenant isolation,
 - IDOR/resource access,
 - storage access,
 - API abuse,
 - WebSocket authorization,
 - secret exposure,
 - input validation.
-

59. IMPLEMENTATION ORDER

The technical implementation should progress in a controlled sequence.

Phase 1 — Foundation

Establish:

- repository,
- frontend,
- backend,
- environment configuration,
- authentication foundation,
- database connection.

Phase 2 — JD Intelligence

Implement:

- JD input,

- parsing,
- requirement extraction,
- rubric generation,
- question generation.

Phase 3 — Interview Engine

Implement:

- interview sessions,
- real-time communication,
- STT,
- LLM,
- TTS,
- adaptive questioning.

Phase 4 — Evaluation & Integrity

Implement:

- transcript processing,
- evaluation,
- scoring,
- communication metrics,
- proctoring.

Phase 5 — Analytics & Reports

Implement:

- recruiter dashboard,
- ranking,
- analytics,
- candidate diagnostic reports.

Phase 6 — Optimization & Deployment

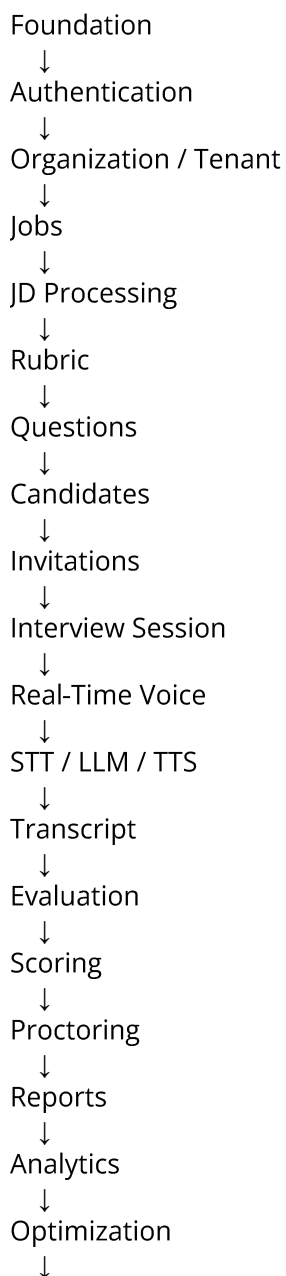
Implement:

- performance optimization,
- scalability,

- security hardening,
 - testing,
 - Docker,
 - deployment.
-

60. IMPLEMENTATION DEPENDENCY FLOW

The recommended dependency sequence is:



61. TECHNICAL IMPLEMENTATION PRINCIPLES

The implementation must follow these principles:

Principle 1 — Separation of Concerns

Each layer has a clear responsibility.

Principle 2 — Domain Orientation

Major functionality should be organized around QualifyAI domains.

Principle 3 — Provider Independence

AI providers should remain replaceable.

Principle 4 — Tenant Isolation

Organization data must remain isolated.

Principle 5 — Secure by Default

Authorization and validation must be enforced server-side.

Principle 6 — Explicit State

Important asynchronous and interview states must be represented explicitly.

Principle 7 — Observable Systems

Critical operations must be traceable.

Principle 8 — Documentation Synchronization

Architectural changes must be reflected in documentation.

62. IMPLEMENTATION BOUNDARIES

The following boundaries must remain clear:

CLIENT



Presentation & Interaction

SERVER



Business Logic & Orchestration

INTEGRATIONS



External Providers

DATABASE



Structured Persistent Data

STORAGE



Files & Media

DOCUMENTATION



System Knowledge & Specifications

63. WHAT SHOULD NOT BE IMPLEMENTED AS A SEPARATE SERVICE

The project should not prematurely create independent microservices for:

- JD parsing,
- rubric generation,
- evaluation,
- scoring,
- proctoring,

unless scaling or deployment requirements later justify such separation.

These can initially remain modular services/modules within the backend.

64. AI PROVIDER ABSTRACTION

The architecture should make it possible to change:

Deepgram
↓
Another STT Provider

or:

OpenAI
↓
Gemini
↓
Another LLM

or:

ElevenLabs
↓
Another TTS Provider

without rewriting the core interview engine.

65. DATA FLOW — RECRUITER

Recruiter
↓
Client
↓
Authenticated API
↓
Job Service
↓
JD Processing
↓
Requirement Extraction
↓
Rubric Generation
↓
Question Generation
↓
Database

↓
Candidate Invitation

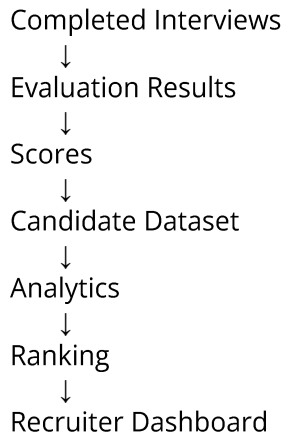
66. DATA FLOW — CANDIDATE

Candidate
↓
Client
↓
Authentication / Interview Authorization
↓
Interview Session
↓
WebSocket
↓
Audio
↓
STT
↓
Interview Engine
↓
LLM
↓
TTS
↓
Candidate

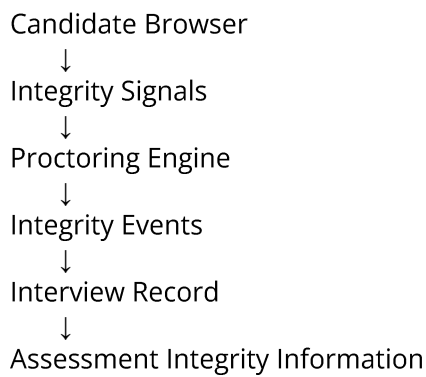
67. DATA FLOW — EVALUATION

Interview
↓
Transcript
↓
Rubric
↓
Evaluation Engine
↓
Technical Evaluation
↓
Communication Metrics
↓
Scoring Engine
↓
Final Evaluation
↓
Report

68. DATA FLOW — RECRUITER ANALYTICS



69. DATA FLOW — PROCTORING



70. FAILURE RECOVERY

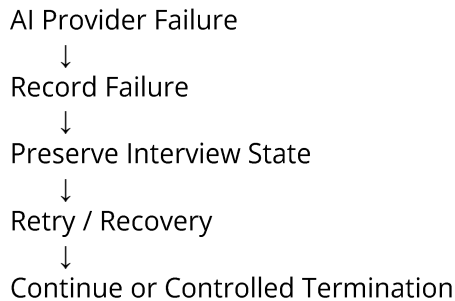
The system must gracefully handle failures in:

- authentication,
- database,
- storage,
- WebSocket,
- STT,
- LLM,
- TTS,
- queue,

- report generation.

The system should preserve valid interview state wherever possible.

For example:



71. DOCUMENTATION REQUIREMENTS

Technical implementation changes must be reflected in the project's documentation system.

Primary documents:

README.md
prd.md
architecture.md
design.md
memory.md
phases.md
rules.md

Supporting technical material belongs in:

docs/

Formal project artifacts belong in:

document/

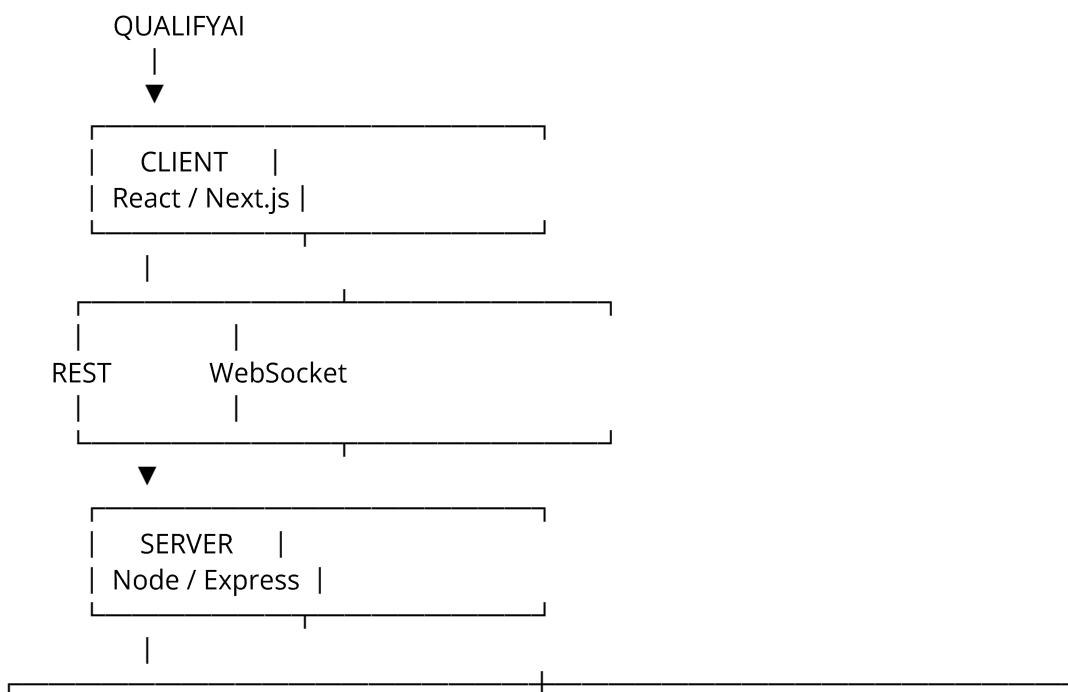
72. IMPLEMENTATION ACCEPTANCE CRITERIA

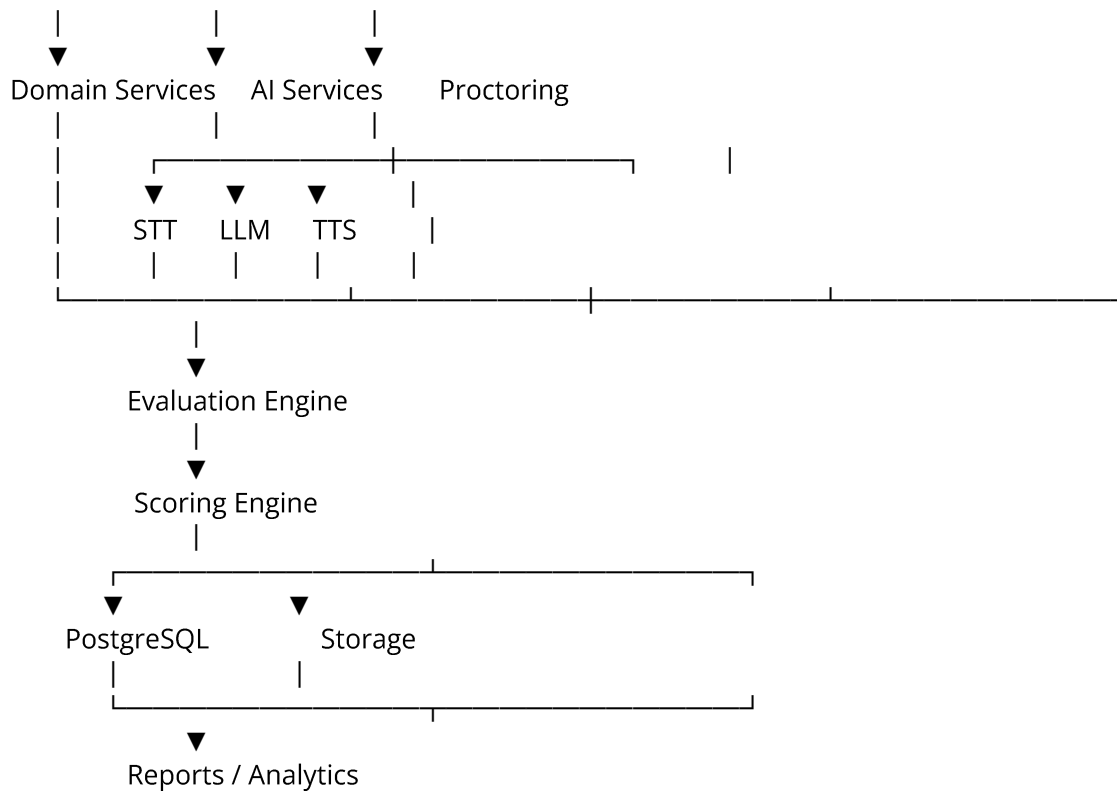
Document 2 implementation planning is considered complete when:

- frontend responsibilities are defined,
- backend responsibilities are defined,
- API boundaries are defined,
- AI boundaries are defined,
- real-time architecture is defined,
- evaluation pipeline is defined,
- proctoring boundary is defined,
- storage architecture is defined,
- database interaction is defined,
- authentication is defined,
- multi-tenancy is defined,
- security boundaries are defined,
- asynchronous processing is defined,
- deployment direction is defined,
- testing direction is defined,
- implementation phases are defined.

73. FINAL TECHNICAL ARCHITECTURE

The complete technical implementation can be summarized as:





74. FINAL IMPLEMENTATION DEFINITION

QualifyAI should be implemented as a **modular, multi-tenant, AI-enabled technical recruitment platform** rather than as a collection of independent AI features.

The central technical principle is:

The Job Description establishes the assessment context; the interview engine conducts the conversation; the evaluation engine measures performance; and the analytics/reporting layer converts the assessment into recruitment intelligence.

The architecture must therefore maintain a continuous relationship between:

```

Job
↓
Requirements
↓
Rubric
↓
Questions
↓
Interview
↓

```

Responses
↓
Evaluation
↓
Score
↓
Report
↓
Recruitment Decision

This relationship is the core technical dependency chain of QualifyAI.

DOCUMENT 2 — END

Document: QualifyAI — Full Technical Implementation Plan

Status: Foundation Technical Specification

Purpose: Defines how the QualifyAI product is technically implemented

Next: Document 3 — QualifyAI Complete Web App Frontend Blueprint